

Name: Paige Millis-Wight

ASU Student ID (10 digit #): 1224482272

Program: 4+1 M.S. Statistics (Applied Track)

Please upload project and any other supporting documentation here:



Start date of MA/MS portfolio (e.g.12/21/2019): 05/06/2026

Project course (e.g. MAT 570): STP 540

If this is a 598 course, please include the full course name: NA

Indicate Semester: Spring Year: 2026 of project course.

Course instructor: Dr. Robert McCulloch

Completion date for course project (e.g.12/21/2019) 05/06/2026

05/06/2026

Paige Millis-Wight
Paige Millis-Wight (May 6, 2026 21:45:59 PDT)

Grade outcome for project (select Pass or Fail): PASS

Instructor's Signature Robert McCulloch
Robert McCulloch (May 8, 2026 07:34:01 PDT)

Date 05/08/2026

Comments for revision. If the project needs some minor revision, please add comments here. The student will be notified to re-submit new form with changes.

May we publish the following project on the school (math.asu.edu) website for other students to view as an example? (Select Yes or No in dropdown)

a. Student: Yes

b. Instructor: Yes

c. Were there other students involved in your project? If so, please LIST each of the student collaborator's name and emails below. All student collaborators will be emailed for approval to publish your project on the website:

List each student who collaborated with you on this project:

Student 1: _____

Email: _____

Student 2: _____

Email: _____

Building a Single Layer Neural Network from Scratch: A Public Health Application

Paige Millis-Wight (1224482272)

Dr. Rob McCulloch

STP 540 — Computational Statistics

Arizona State University

May 7, 2026

1 Introduction

Machine learning and artificial intelligence have become central tools in data science, yet their underlying mechanics remain opaque to many practitioners who rely on library calls and blackbox implementations. This project takes a different approach by building a single layer neural network entirely from scratch in R, without Keras, PyTorch, or any machine learning library. The goal is not merely to produce predictions, but to demonstrate understanding of the statistical and computational principles that make neural networks work.

The application is systolic blood pressure prediction using behavioral and demographic predictors from the National Health and Nutrition Examination Survey (NHANES) 2017–2018. Blood pressure is the leading modifiable cardiovascular risk factor in the United States, and understanding its behavioral determinants has direct implications for public health policy. However, the choice of application is secondary to the methodological goal which is to create a general purpose neural network built from mathematical first principles can be applied to any dataset. Understanding why it works, when it fails, and what its limitations mean is more valuable than any single result.

Neural networks are the architectural foundation of modern machine learning and artificial intelligence. A single layer network is the building block from which deep learning systems are constructed. Every concept in deep learning, from convolutional networks for image recognition to large language models, traces back to the forward pass, backpropagation, and gradient descent implemented here. Building it from scratch emphasizes the difference between knowing how to call a function and understanding what that function is doing.

The instinct that motivates this work extends beyond blood pressure prediction. Pattern recognition across complex systems, whether biological, economic, or historical, is fundamentally the same problem. A neural network predicting blood pressure from behavioral variables is doing the same thing, mechanically, as a model predicting economic instability from political indicators, or epidemic emergence from environmental and demographic sig-

nals. The tool is general. Domain knowledge determines whether the results are meaningful.

2 Literature Review

2.1 The Global Burden of Hypertension

Hypertension represents one of the most significant preventable causes of morbidity and mortality worldwide. An estimated 1.28 billion adults globally are affected, with the burden disproportionately concentrated in low- and middle-income countries [1]. In 2015 alone, approximately 8.5 million deaths were attributable to elevated systolic blood pressure exceeding 115 mmHg, with 88% occurring in low- and middle-income settings [1]. In the United States, hypertension remains the leading modifiable risk factor for ischemic heart disease, stroke, chronic kidney disease, and dementia, making its prediction and prevention a central public health priority.

2.2 Behavioral Determinants of Hypertension

While genetic and physiological factors contribute to hypertension onset, behavioral determinants are particularly actionable targets for public health intervention. Sullivan et al. [2] provide a comprehensive review of behavioral risk factors including dietary patterns, physical activity, stress, sleep quality, and treatment adherence, demonstrating through longitudinal evidence that sustained behavioral interventions can reduce both the incidence and severity of hypertension. Notably, lifestyle modifications alone have been shown to reduce systolic blood pressure by 4–10 mmHg in prehypertensive individuals [2]. The Dietary Approaches to Stop Hypertension (DASH) trial demonstrated an average systolic blood pressure reduction of 11 mmHg through dietary intervention alone, establishing that behavioral factors have clinically meaningful independent effects on blood pressure.

Age and BMI are among the most consistently documented behavioral and demographic

predictors of hypertension. Body mass index reflects the cumulative effect of dietary and activity patterns on adiposity, with excess weight increasing vascular resistance and cardiac output. Age captures the progressive arterial stiffening that accompanies biological aging, independent of lifestyle factors. Sleep duration and quality have more recently emerged as significant predictors, with both short and long sleep associated with elevated blood pressure through neuroendocrine and inflammatory pathways [2].

2.3 Machine Learning in Cardiovascular Health

The application of machine learning to cardiovascular risk prediction has expanded substantially in recent years. Bhowmik et al. [3] demonstrated that machine learning algorithms including logistic regression, random forest, and support vector machines can effectively predict heart disease from clinical features, with logistic regression achieving the highest ROC-AUC score in their comparison. Their findings highlight a recurring theme in applied cardiovascular machine learning. This being that simpler, more interpretable models often perform comparably or superiorly to complex ones, particularly when training data is limited or predictor sets are incomplete.

Neural networks offer particular advantages when relationships between predictors and outcomes are nonlinear or involve complex interactions that linear models cannot capture. However, as James et al. [5] note, neural networks do not universally outperform simpler methods. When used on structured tabular data with limited predictors, regularized linear models frequently achieve comparable or better out-of-sample performance with substantially greater interpretability. This tradeoff between flexibility and interpretability is especially consequential in health applications where clinicians and policymakers require models they can understand and act upon.

2.4 Confounding in Healthcare Database Research

A persistent methodological challenge in blood pressure prediction using population health databases is unmeasured confounding. Brookhart et al. [4] identify the absence of key clinical variables as a fundamental limitation of healthcare database research, noting that exposure is often determined by complex interactions of patient, physician, and healthcare system factors that are not captured in administrative or survey data. This challenge is particularly acute for blood pressure prediction, where antihypertensive medication use represents a major unmeasured confounder. Individuals with the highest underlying cardiovascular risk may present with controlled blood pressure readings due to pharmacological treatment, systematically attenuating the apparent relationship between behavioral risk factors and observed blood pressure values. Brookhart et al. recommend reporting results under various model specifications to assess sensitivity to these unmeasured confounders, an approach that motivates the multi-model comparison strategy employed in this analysis.

3 Methods

3.1 The Single Layer Neural Network Model

A single layer neural network predicts a continuous outcome Y from a vector of predictors $\mathbf{X} = (X_1, X_2, \dots, X_p)$ through three sequential operations.

Step 1 — Linear combinations. For each of K hidden units, a linear combination of the predictors is computed:

$$z_k = w_{k0} + \sum_{j=1}^p w_{kj}X_j, \quad k = 1, 2, \dots, K \quad (1)$$

where w_{k0} is a bias term and w_{kj} are input weights. Each unit sees all predictors simultaneously, producing K different linear projections of the input space.

Step 2 — Activation. Each linear combination is passed through a nonlinear activation function. The sigmoid function was used throughout:

$$g(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

The sigmoid maps any real number to the interval $(0, 1)$, producing an S-shaped curve. By changing the input weights, each unit produces a differently positioned, scaled, and oriented sigmoid surface over the predictor space. Adding K such surfaces together can approximate virtually any smooth function via the universal approximation theorem.

Step 3 — Output. The K activations are combined linearly to produce the final prediction:

$$\hat{y} = \beta_0 + \sum_{k=1}^K \beta_k A_k \quad (3)$$

where β_0 is an output intercept and β_k are output weights. The complete model in one expression:

$$\hat{y} = \beta_0 + \sum_{k=1}^K \beta_k g\left(w_{k0} + \sum_{j=1}^p w_{kj} X_j\right) \quad (4)$$

3.2 Why Nonlinearity Matters

Linear regression predicts Y as a linear combination of predictors, a flat plane through the predictor space. No matter how many predictors are added, a linear combination of linear functions remains linear. This is fundamentally limiting when relationships between predictors and outcomes are curved, interactive, or otherwise nonlinear.

The neural network breaks this constraint through the activation function. Each unit bends a linear projection into a curved surface. The output layer adds these curved surfaces together. The result is a flexible nonlinear function that can capture relationships linear regression

cannot see. To demonstrate this concretely, the network was first tested on simulated data where the true relationship is known: $y = x^2 + \varepsilon$. A straight line cannot fit a parabola. The neural network, by adding sigmoid curves together, can.

3.3 Architecture and the Role of Hidden Units

The number of hidden units K controls model flexibility. With too few units the model lacks the capacity to capture complex patterns. With too many units the model may memorize noise in the training data rather than learning generalizable patterns, this is called overfitting.

Thinking about this in terms of layers is helpful. A single hidden layer makes one level of transformation by taking raw predictors and finding combinations useful for predicting the outcome. With richer data and more complex relationships, multiple hidden layers could build hierarchical representations. In the context of blood pressure prediction, a deeper network with complete clinical data might learn in its first layer that high BMI combined with low physical activity constitutes a metabolic risk pattern, and in its second layer that this pattern interacts differently with age in younger versus older adults. With only three behavioral predictors, a single hidden layer is appropriate as the relationships are not hierarchically complex enough to justify deeper architecture.

3.4 Loss Function and Training

The network is trained by minimizing mean squared error loss:

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5)$$

Training proceeds by iteratively adjusting all weights in the direction that reduces loss. The gradient of the loss with respect to each weight is computed using the chain rule, moving backward through the network from the loss to the input weights. This process is known as

backpropagation.

For the output weights:

$$\frac{\partial L}{\partial \beta_k} = -2(y - \hat{y}) A_k \quad (6)$$

For the input weights:

$$\frac{\partial L}{\partial w_{kj}} = -2(y - \hat{y}) \beta_k g'(z_k) X_j \quad (7)$$

where $g'(z) = g(z)(1 - g(z))$ is the sigmoid derivative. The four terms in the input weight gradient have intuitive interpretations. The error term $-2(y - \hat{y})$ captures how wrong the prediction was. The output weight β_k captures how much this unit influenced the final prediction. The sigmoid derivative $g'(z_k)$ captures how sensitive the activation currently is to weight changes. More abstractly, whether the dial is in a position where turning it actually does anything. The predictor value X_j captures how much this input was involved. Together they answer how much should this weight change, given what happened.

3.5 Stochastic Gradient Descent

Computing the full gradient across all n observations before each weight update is computationally expensive for large datasets. Stochastic gradient descent (SGD) addresses this by computing gradients on small random subsets of data called minibatches. One complete pass through all minibatches constitutes an epoch. The weight update at each minibatch of size m is:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \frac{1}{m} \sum_{i=1}^m \nabla L(x_i^b, y_i^b, \boldsymbol{\theta}) \quad (8)$$

where α is the learning rate and θ represents all weights and biases.

3.6 L2 Regularization

To prevent overfitting, an L2 penalty was added to the loss function:

$$L_{\text{reg}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_j \theta_j^2 \quad (9)$$

where θ represents all weights and λ controls regularization strength. The gradient update adds $2\lambda\theta_j$ to each weight gradient, shrinking weights toward zero and encouraging simpler solutions that generalize better to new data.

3.7 Data

Simulated data. To verify the network implementation before applying it to real data, 100 observations were simulated from $y = x^2 + \varepsilon$ where $x \sim \text{Uniform}(-3, 3)$ and $\varepsilon \sim \mathcal{N}(0, 0.25)$. This nonlinear relationship cannot be captured by linear regression, providing a clean verification test.

NHANES data. The National Health and Nutrition Examination Survey 2017–2018 cycle was accessed using the `nhanesA` package in R. Systolic blood pressure was computed as the average of up to four repeated readings per participant (BPXSY1–BPXSY4), consistent with clinical measurement standards. Predictors included age (RIDAGEYR), BMI (BMXBMI), and sleep hours (SLD012). Physical activity and alcohol use were excluded due to greater than 75% missing data. After removing observations with missing values on any retained variable, the analytic sample included 5,142 adults aged 18–80. All predictors were standardized to mean zero and unit variance prior to model fitting. A 75/25 train/test split was used for all model evaluation.

4 Results

4.1 Simulated Data: Proof of Concept

Training dynamics. Starting from random weights, the initial mean squared error loss was 13.684. After 1,000 epochs of gradient descent the loss fell to 0.295, a 97.8% reduction. The loss curve showed steep initial decline followed by flattening around epoch 300, suggesting convergence well before the epoch limit. In practice, early stopping at epoch 300 would have produced equivalent results with less computation.

Effect of hidden units and regularization.

Table 1: Simulated Data Model Comparison

Model	Train MSE	Test MSE
$K = 2$, no regularization	0.328	0.220
$K = 2$, L2 regularization	0.665	0.656
$K = 10$, no regularization	0.338	0.292
$K = 10$, L2 regularization	0.525	0.500

The simplest model, $K = 2$ with no regularization, achieved the best test performance. A parabola is a relatively simple function that two sigmoid units can approximate well. Adding more units introduces parameters that find noise rather than signal. Applying regularization to $K = 2$ over constrained an already simple model. These results illustrate that optimal architecture depends on the complexity of the underlying relationship, not just on maximizing model capacity.

Comparison to linear regression. Linear regression achieved a test MSE of 9.323, this is 42 times worse than the best neural network (MSE = 0.220).

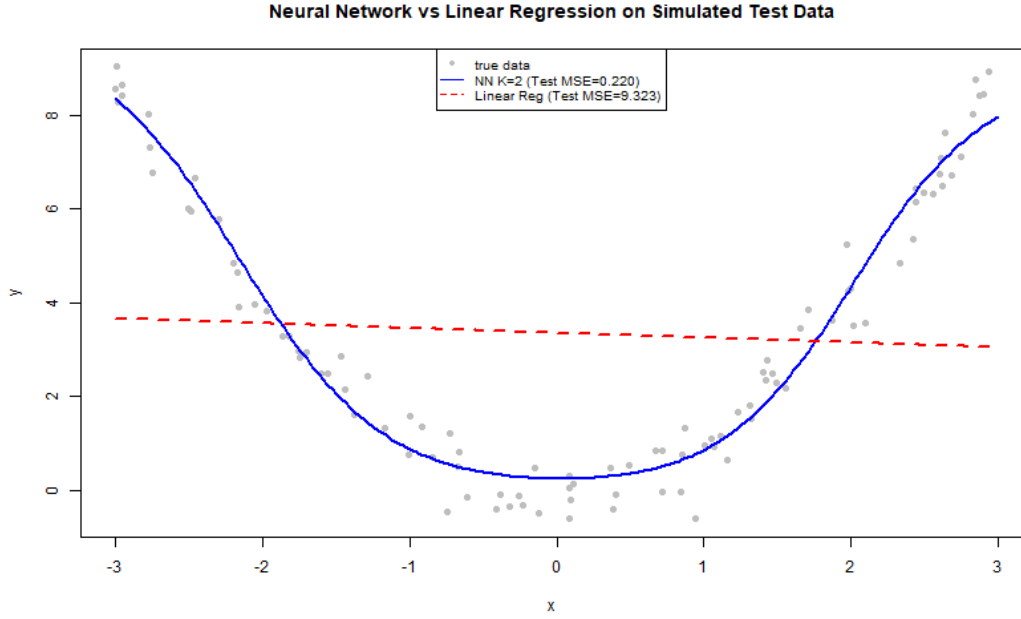


Figure 1: Neural network versus linear regression fit on simulated test data ($y = x^2 + \varepsilon$). The neural network (blue, solid) captures the nonlinear U-shaped relationship closely while linear regression (red, dashed) fits a flat line through the data, demonstrating the fundamental limitation of linear models on nonlinear relationships. Test MSE: NN = 0.220, Linear Regression = 9.323.

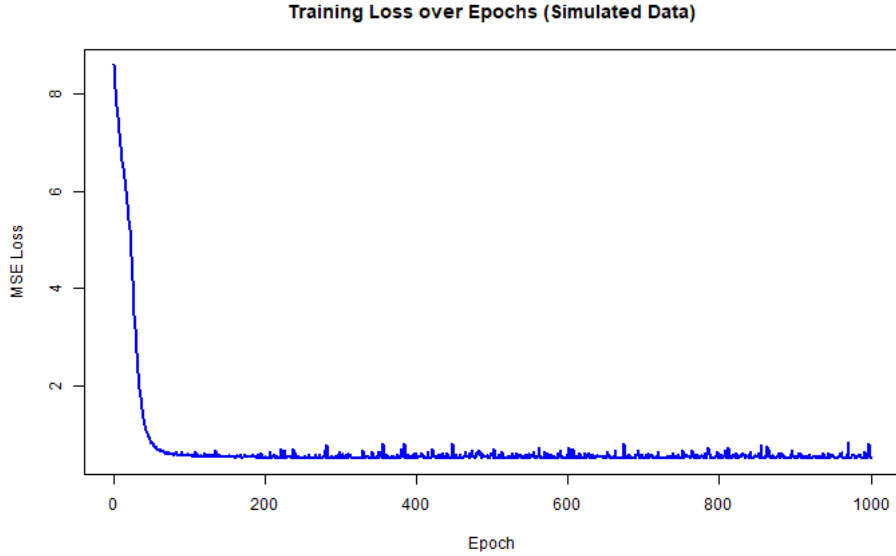


Figure 2: Training loss over 1,000 epochs on simulated data. Loss drops steeply in the first 50 epochs then flattens, suggesting convergence around epoch 300. Persistent minor fluctuations reflect the stochastic nature of minibatch gradient descent.

4.2 NHANES Application: Predicting Systolic Blood Pressure

Table 2: NHANES Model Comparison (Systolic Blood Pressure, mmHg²)

Model	Train MSE	Test MSE
Linear Regression	302.847	261.978
NN $K = 2$, no regularization	318.376	284.767
NN $K = 10$, no regularization	326.816	289.421
NN $K = 10$, L2 regularization	297.306	259.760
NN $K = 25$, L2 regularization	292.390	258.692

The best neural network, $K = 25$ with L2 regularization, achieved a test MSE of 258.692, a 1.3% improvement over linear regression. This is substantially smaller than the 97.8% improvement seen on simulated data. Loss curves showed persistent noise characteristic of SGD on a larger, noisier dataset. Predicted versus actual plots revealed systematic compression toward the center of the distribution, with very high blood pressure readings (150–200 mmHg) consistently underestimated by both models.

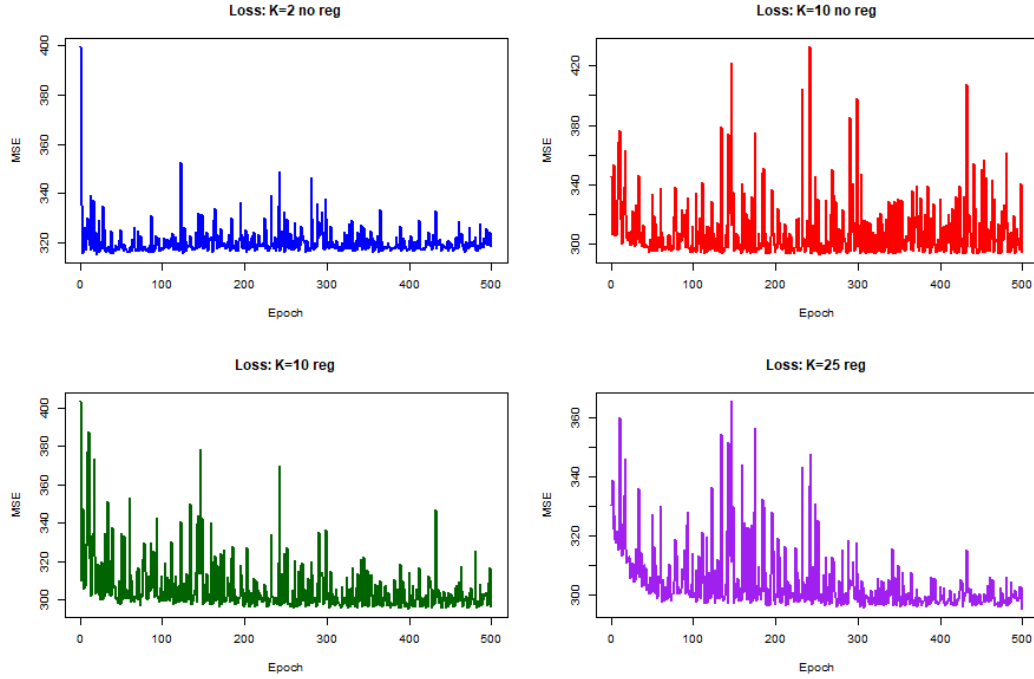


Figure 3: Training loss curves over 500 epochs for all four neural network configurations on NHANES data. All models show an initial decline followed by stable fluctuation characteristic of SGD on large, noisy datasets. Regularized models (K=10 reg, K=25 reg) show more stable convergence than unregularized models.

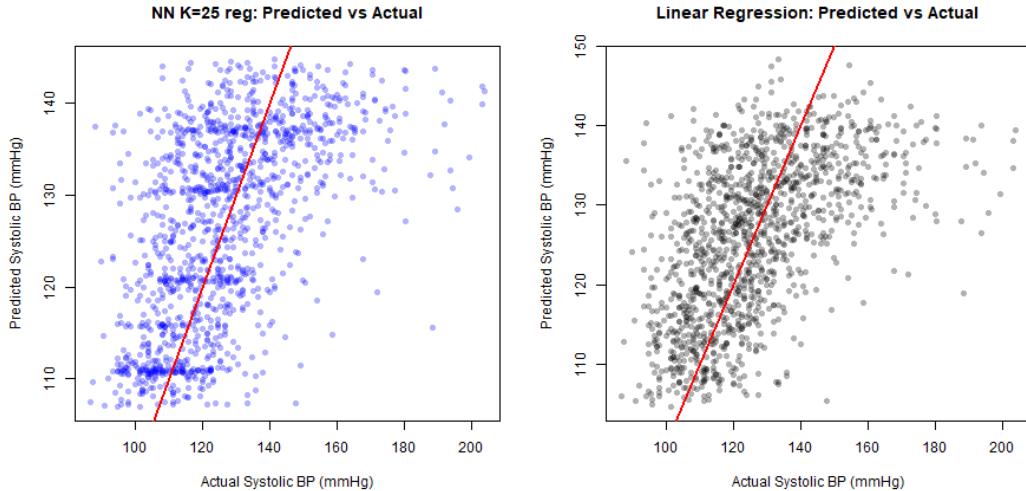


Figure 4: Predicted versus actual systolic blood pressure on NHANES test data ($n=1,286$). The red diagonal line represents perfect prediction. Both the neural network ($K = 25$, L2 regularization, left) and linear regression (right) show compression toward the center of the distribution, systematically underestimating very high blood pressure readings (150–200 mmHg). This pattern is consistent with unmeasured confounding by antihypertensive medication use [4].

5 Discussion

5.1 When Neural Networks Win and When They Don't

The contrast between simulated and NHANES results tells us something important. On simulated data with a known nonlinear relationship, the neural network dramatically outperformed linear regression. On real blood pressure data with three behavioral predictors, the improvement was negligible.

This is not a failure of the neural network but it is a finding about the data. The near-equivalence of linear regression and neural network performance suggests the relationship between age, BMI, sleep, and systolic blood pressure is approximately linear within this

sample. The neural network’s nonlinear flexibility provided little benefit because the underlying relationship did not require it. This echoes the broader principle that neural networks do not always win. The appropriate tool depends on the structure of the problem, not the sophistication of the method [5].

5.2 The Data Ceiling Problem

The convergence of test MSE across K values suggests the ceiling is imposed by data limitations rather than model capacity. Antihypertensive medication use is probably the largest omitted variable, a person on three blood pressure medications may have a controlled reading of 118 mmHg despite underlying cardiovascular risk that would otherwise produce readings of 160 or higher. The model sees 118 and learns nothing about their true risk profile. Genetic factors, dietary sodium, smoking, stress, and existing cardiovascular disease are similarly absent.

Building a neural network with $K = 1000$ units would not solve this problem. More parameters give the model more capacity to find complex patterns, but if the relevant patterns are not in the predictor set, additional capacity only finds more elaborate ways to fit noise. The bottleneck is information, not flexibility.

5.3 What Deeper Layers Could Tell Us

A useful analogy for understanding multiple layers is visual recognition. Recognizing an apple requires multiple levels of abstraction: edges and colors at the first layer, shapes at the second, object categories at the third, specific identification at the fourth. No single layer can jump from raw pixels to “apple.” Similarly, a single layer predicting blood pressure from three behavioral predictors makes one level of transformation. With complete clinical data, deeper layers could build richer representations: the first layer identifying metabolic risk patterns from BMI, physical activity, and sodium; the second learning how those patterns

interact with age and demographics; the third identifying clinically actionable subgroups for targeted intervention. The gap between what we know clinically and what we can model computationally is fundamentally a data availability problem, not a modeling problem.

5.4 Implications for Health Policy and Consulting

The honest result of this analysis, that three behavioral predictors explain limited variance in blood pressure, is itself policy relevant. It quantifies what epidemiologists have long argued qualitatively, that behavioral interventions targeting BMI and sleep operate against a backdrop of genetic, structural, and pharmaceutical factors that dwarf their individual effects in population-level data.

A model with RMSE of approximately 16 mmHg is not clinically actionable at the individual level, this error spans the difference between normal and hypertensive classifications. But at the population level, identifying that age and BMI explain meaningful variance in blood pressure distribution can inform resource allocation, screening priorities, and intervention targeting even when individual predictions are imprecise.

The more powerful consulting application would integrate electronic health records including medication data, laboratory values, and longitudinal measurements. With that data, the neural network architecture built here could be applied directly, the functions require only a different $\mathbf{X}$ matrix. The infrastructure is general. The limitation was always the data.

5.5 Broader Reflections on Pattern Recognition Across Complex Systems

The mechanics of learning by iteratively adjusting parameters to minimize prediction error across many observations are domain agnostic. The same architecture predicting blood pressure from behavioral variables could predict political instability from economic indicators, epidemic emergence from environmental signals, or economic crashes from market

microstructure data.

This is not only theoretical. Researchers in cliodynamics, which is the mathematical modeling of historical dynamics, have applied quantitative pattern recognition to centuries of historical data, identifying cycles in political violence, elite overproduction, and institutional trust that recur across civilizations [7]. The insight that history follows learnable patterns is the same insight that underlies neural network prediction. Complex systems, despite their apparent chaos, contain structure that iterative optimization can find.

The difference between a data scientist and a health data consultant is not technical sophistication. It is the ability to recognize which patterns are worth finding, what data would be needed to find them, and what finding them would mean for decisions that affect people. The neural network is a tool. The judgment about where to point it, and what to do with what it finds, remains irreducibly human.

5.6 Limitations

Several limitations warrant acknowledgment. First, the exclusion of physical activity and alcohol use due to missing data removed potentially important predictors, imputation methods could partially address this. Second, the cross-sectional design of NHANES precludes causal inference. Third, the network was trained with a fixed learning rate; implementing rmsprop or Adam optimization would likely improve convergence stability. Fourth, hyperparameter selection was done by comparison rather than systematic cross-validation.

5.7 Next Steps

Immediate improvements would include adding antihypertensive medication status as a predictor, which would likely reduce prediction error substantially by separating controlled from uncontrolled hypertension. Expanding the predictor set to include dietary sodium, smoking status, and physical activity with proper missing data handling would further improve

performance. Implementing adaptive learning rates and systematic hyperparameter tuning via cross-validation would improve training stability. Finally, applying the architecture to longitudinal NHANES data would enable prediction of blood pressure trajectories. This is a more clinically meaningful target than a single time point measurement.

6 Conclusion

This project built a single layer neural network from mathematical first principles: forward pass, backpropagation, stochastic gradient descent, and L2 regularization. These principles were implemented entirely in R without machine learning libraries. On simulated nonlinear data the network outperformed linear regression by 97.8%, demonstrating that the implementation is correct and that neural networks provide genuine advantages when relationships are nonlinear. On real NHANES blood pressure data the improvement over linear regression was marginal. This finding reflects data limitations rather than model failure, specifically the absence of medication use, genetic factors, and complete dietary data.

The technical contribution is a general purpose neural network applicable to any continuous prediction problem by substituting a different data matrix. The intellectual contribution is understanding when that tool is appropriate, what its results mean, and what its limitations imply for the questions we can and cannot answer with available data. In health consulting, that understanding is more valuable than any single model result.

References

- [1] Zhou, B., Perel, P., Mensah, G. A., & Ezzati, M. (2021). Global epidemiology, health burden and effective interventions for elevated blood pressure and hypertension. *Nature Reviews Cardiology*, 18, 785–802. <https://doi.org/10.1038/s41569-021-00559-8>

- [2] Sullivan, H., Coleman, P., Richardson, F., & Bennett, V. (2026). Behavioral determinants of hypertension: A long-term public health perspective. Unpublished manuscript.
- [3] Bhowmik, P. K., Miah, M. N. I., Uddin, M. K., Sizan, M. M. H., Pant, L., Islam, M. R., & Gurung, N. (2024). Advancing heart disease prediction through machine learning: Techniques and insights for improved cardiovascular health. *British Journal of Nursing Studies*, 4(2). <https://doi.org/10.32996/bjns.2024.4.2.5>
- [4] Brookhart, M. A., Stürmer, T., Glynn, R. J., Rassen, J., & Schneeweiss, S. (2010). Confounding control in healthcare database research: Challenges and potential approaches. *Medical Care*, 48(6), S114–S120. <https://doi.org/10.1097/MLR.0b013e3181dbebe3>
- [5] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning, with Applications in R* (2nd ed.). Springer.
- [6] Kolar, M., & McCulloch, R. (2026). *STP 540 Course Notes: Neural Networks and Optimization*. Arizona State University.
- [7] Turchin, P. (2023). *End Times: Elites, Counter-Elites, and the Path of Political Disintegration*. Penguin Press.
- [8] Centers for Disease Control and Prevention. (2018). *National Health and Nutrition Examination Survey 2017–2018*. U.S. Department of Health and Human Services.
- [9] Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.

A R Code

All analysis was conducted in R. The complete implementation is provided below, including data simulation, neural network functions, NHANES data acquisition, model training, and visualization.

A.1 Neural Network Functions

```
#####  
### Single Layer Neural Network -- From Scratch  
### STP 540 Final Project  
### Paige Millis-Wight  
#####  
  
#####  
### Activation Function  
#####  
  
sigmoid = function(z) {  
  1 / (1 + exp(-z))  
}  
  
#####  
### Forward Pass  
### X: n x (p+1) design matrix with column of 1s  
### W: K x (p+1) input weight matrix  
### beta: (K+1) output weight vector  
#####
```

```

forward_pass = function(X, W, beta) {

  ## step 1: linear combinations
  ## Z is K x n
  Z = W %*% t(X)

  ## step 2: activation
  ## A is K x n
  A = sigmoid(Z)

  ## step 3: output
  ## add row of 1s for output intercept
  A_aug = rbind(1, A)

  ## yhat is n x 1
  yhat = t(A_aug) %*% beta

  return(list(yhat=yhat, Z=Z, A=A))
}

#####

### Backward Pass

### Computes gradients via chain rule

#####

backward_pass = function(X, ydat, yhat, Z, A, W, beta) {

```

```

nn = nrow(X)
K = nrow(W)

## error
error = ydat - yhat

## sigmoid derivative:  $F'(z) = A(1-A)$ 
Fprime = A * (1 - A)

## gradient for output weights
A_aug = rbind(1, A)
grad_beta = -2 * (A_aug %*% error) / nn

## gradient for input weights
beta_noint = beta[-1]
delta = (beta_noint %*% t(error)) * Fprime
grad_W = -2 * (delta %*% X) / nn

return(list(grad_beta=grad_beta, grad_W=grad_W))
}

#####

### Training Loop with SGD and L2 Regularization
### K: number of hidden units
### learning_rate: step size alpha
### n_epochs: number of full passes through data

```

```

### batch_size: minibatch size

### lambda: L2 regularization strength

#####

train_nn_sgd = function(X, ydat,

                        K = 5,

                        learning_rate = 0.1,

                        n_epochs = 1000,

                        batch_size = 20,

                        lambda = 0.001) {

  nn = nrow(X)
  p = ncol(X) - 1

  ## random weight initialization
  set.seed(42)
  W = matrix(rnorm(K*(p+1)), nrow=K, ncol=p+1)
  beta = rnorm(K+1)

  loss_history = rep(0, n_epochs)

  for(epoch in 1:n_epochs) {

    ## shuffle data
    shuffle_idx = sample(1:nn)
    X_shuf = X[shuffle_idx, ]
    y_shuf = ydat[shuffle_idx]

```



```

n_batches = floor(nn / batch_size)

for(k in 1:n_batches) {

  batch_idx = ((k-1)*batch_size + 1):(k*batch_size)
  X_batch = X_shuf[batch_idx, ]
  y_batch = y_shuf[batch_idx]

  result = forward_pass(X_batch, W, beta)
  yhat_batch = result$yhat

  grad = backward_pass(X_batch, y_batch,
                        yhat_batch,
                        result$Z, result$A,
                        W, beta)

  ## add L2 penalty to gradients
  grad$grad_W = grad$grad_W + 2*lambda*W
  grad$grad_beta[-1] = grad$grad_beta[-1] +
                        2*lambda*beta[-1]

  ## update weights
  beta = beta - learning_rate * grad$grad_beta
  W = W - learning_rate * grad$grad_W
}

```

```

    ## compute full data loss
    result_full = forward_pass(X, W, beta)
    yhat_full = result_full$yhat
    mse = mean((ydat - yhat_full)^2)
    penalty = lambda * (sum(W^2) + sum(beta[-1]^2))
    loss_history[epoch] = mse + penalty
  }

  result_final = forward_pass(X, W, beta)
  yhat_final = result_final$yhat

  return(list(W=W, beta=beta,
             yhat=yhat_final,
             loss_history=loss_history))
}

```

A.2 Simulated Data Analysis

```

#####

### Simulated Data:  $y = x^2 + \text{noise}$ 

#####

set.seed(42)

n = 100

xdat = runif(n, -3, 3)
ydat = xdat^2 + rnorm(n, 0, 0.5)

## design matrix with intercept

```

```

X = cbind(1, xdat)

## train/test split
set.seed(42)
train_idx = sample(1:n, floor(0.75*n))
test_idx  = setdiff(1:n, train_idx)

X_train = X[train_idx, ]; y_train = ydat[train_idx]
X_test  = X[test_idx, ]; y_test  = ydat[test_idx]

## train four model configurations
nn_K2 = train_nn_sgd(X_train, y_train, K=2,
                     learning_rate=0.1,
                     n_epochs=500, batch_size=20,
                     lambda=0.0)

nn_K2_reg = train_nn_sgd(X_train, y_train, K=2,
                         learning_rate=0.1,
                         n_epochs=500, batch_size=20,
                         lambda=0.01)

nn_K10 = train_nn_sgd(X_train, y_train, K=10,
                      learning_rate=0.1,
                      n_epochs=500, batch_size=20,
                      lambda=0.0)

nn_K10_reg = train_nn_sgd(X_train, y_train, K=10,

```

```

        learning_rate=0.1,
        n_epochs=500, batch_size=20,
        lambda=0.01)

## linear regression baseline
train_df = data.frame(y=y_train, x=xdat[train_idx])
test_df  = data.frame(y=y_test,  x=xdat[test_idx])
lm_fit = lm(y ~ x, data=train_df)
yhat_lm_test = predict(lm_fit, newdata=test_df)
lm_test_mse = round(mean((test_df$y - yhat_lm_test)^2),3)

## test MSE comparison
get_test_mse = function(model, X_te, y_te) {
  result = forward_pass(X_te, model$W, model$beta)
  round(mean((y_te - result$yhat)^2), 3)
}

cat("=== Simulated Data Results ===\n")
cat("Linear Regression Test MSE:", lm_test_mse, "\n")
cat("NN K=2  no reg Test MSE:",
    get_test_mse(nn_K2, X_test, y_test), "\n")
cat("NN K=2  reg   Test MSE:",
    get_test_mse(nn_K2_reg, X_test, y_test), "\n")
cat("NN K=10 no reg Test MSE:",
    get_test_mse(nn_K10, X_test, y_test), "\n")
cat("NN K=10 reg   Test MSE:",
    get_test_mse(nn_K10_reg, X_test, y_test), "\n")

```

A.3 NHANES Data Acquisition and Cleaning

```
#####
```

```
### NHANES Data
```

```
#####
```

```
library(nhanesA)
```

```
## pull relevant tables
```

```
bp = nhanes("BPX_J")
```

```
demo = nhanes("DEMO_J")
```

```
bmx = nhanes("BMX_J")
```

```
slq = nhanes("SLQ_J")
```

```
## select columns
```

```
bp_clean = bp[, c("SEQN", "BPXSY1", "BPXSY2",  
                  "BPXSY3", "BPXSY4")]
```

```
demo_clean = demo[, c("SEQN", "RIDAGEYR")]
```

```
bmx_clean = bmx[, c("SEQN", "BMXBMI")]
```

```
slq_clean = slq[, c("SEQN", "SLD012")]
```

```
## average systolic readings
```

```
bp_clean$systolic = rowMeans(  
  bp_clean[, c("BPXSY1", "BPXSY2", "BPXSY3", "BPXSY4")],  
  na.rm=TRUE)
```

```
bp_clean = bp_clean[, c("SEQN", "systolic")]
```

```
## merge
```

```

dat = merge(bp_clean, demo_clean, by="SEQN")
dat = merge(dat, bmx_clean, by="SEQN")
dat = merge(dat, slq_clean, by="SEQN")
names(dat) = c("SEQN","systolic","age","bmi","sleep")

## remove missing
dat_clean = na.omit(dat[,c("SEQN","systolic",
                           "age","bmi","sleep")])
cat("Final sample size:", nrow(dat_clean), "\n")

## standardize predictors
dat_clean$age_s = scale(dat_clean$age)
dat_clean$bmi_s = scale(dat_clean$bmi)
dat_clean$sleep_s = scale(dat_clean$sleep)

## build X and y
X_nhanes = cbind(1, dat_clean$age_s,
                  dat_clean$bmi_s,
                  dat_clean$sleep_s)
y_nhanes = dat_clean$systolic

## train/test split
set.seed(42)
nn_obs = nrow(X_nhanes)
train_idx_n = sample(1:nn_obs, floor(0.75*nn_obs))
test_idx_n = setdiff(1:nn_obs, train_idx_n)

```

```

X_tr = X_nhanes[train_idx_n, ]
y_tr = y_nhanes[train_idx_n]
X_te = X_nhanes[test_idx_n, ]
y_te = y_nhanes[test_idx_n]

```

A.4 NHANES Model Training and Evaluation

```
#####
```

```
### Train and Compare Models
```

```
#####
```

```

nhanes_K2 = train_nn_sgd(X_tr, y_tr, K=2,
                        learning_rate=0.1,
                        n_epochs=500, batch_size=32,
                        lambda=0.0)

```

```

nhanes_K10 = train_nn_sgd(X_tr, y_tr, K=10,
                        learning_rate=0.1,
                        n_epochs=500, batch_size=32,
                        lambda=0.0)

```

```

nhanes_K10_reg = train_nn_sgd(X_tr, y_tr, K=10,
                        learning_rate=0.1,
                        n_epochs=500, batch_size=32,
                        lambda=0.001)

```

```

nhanes_K25_reg = train_nn_sgd(X_tr, y_tr, K=25,
                        learning_rate=0.1,

```

```

                                n_epochs=500, batch_size=32,
                                lambda=0.001)

## linear regression baseline
nhanes_df_tr = data.frame(y=y_tr,
                           age=X_tr[,2],
                           bmi=X_tr[,3],
                           slp=X_tr[,4])

nhanes_df_te = data.frame(y=y_te,
                           age=X_te[,2],
                           bmi=X_te[,3],
                           slp=X_te[,4])

lm_nhanes = lm(y ~ age + bmi + slp,
               data=nhanes_df_tr)

yhat_lm_te = predict(lm_nhanes, newdata=nhanes_df_te)

lm_mse = round(mean((y_te - yhat_lm_te)^2), 3)

## results table
get_test_mse2 = function(model, X_te, y_te) {
  result = forward_pass(X_te, model$W, model$beta)
  round(mean((y_te - result$yhat)^2), 3)
}

cat("=== NHANES Model Comparison ===\n")
cat("Linear Regression Test MSE:", lm_mse, "\n")
cat("NN K=2  no reg:", get_test_mse2(nhanes_K2,
                                     X_te, y_te), "\n")

```



```
cat("NN K=10 no reg:", get_test_mse2(nhanes_K10,  
                                     X_te, y_te), "\n")  
cat("NN K=10 reg:  ", get_test_mse2(nhanes_K10_reg,  
                                     X_te, y_te), "\n")  
cat("NN K=25 reg:  ", get_test_mse2(nhanes_K25_reg,  
                                     X_te, y_te), "\n")
```